Sistema FCT



Guía de incorporación y operaciones de desarrollo para el Sistema FCT, una aplicación web híbrida que gestiona prácticas de Formación en Centros de Trabajo (FCT).

> Docker Compose es el entorno de desarrollo recomendado, no una receta de producción. Antes de entregar el sistema a otra organización, sigue la guía de puesta en producción: incluye requisitos de infraestructura, secretos, migraciones y, especialmente, la configuración real de AutoFirma y su almacén de confianza.

Inicio rápido

Con Git y Docker Desktop instalados, cuatro comandos dejan la aplicación funcionando en http://localhost:8080:

```
git clone https://github.com/campusedx/Practicas_Desarrollo.git
Set-Location .\Practicas_Desarrollo
Copy-Item .env.example .env
docker compose up -d --build
```

Después, opcionalmente, crea cuentas demo (ver sección 5) y consulta el estado de los servicios con `docker compose ps`.

Índice

1. Tecnologías
2. Requisitos previos
3. Preparar el entorno
    - Puesta en producción
4. Iniciar y detener la infraestructura
5. Base de datos, cuentas de desarrollo y phpMyAdmin
6. Recorrido funcional
7. Acceso a la aplicación y herramientas
8. Frontend y Vite HMR
9. Portal de documentación VitePress
10. Verificaciones y pruebas
11. Solución de problemas frecuentes

1. Tecnologías

- **Backend:** PHP 8.2, Apache y arquitectura MVC personalizada.
- **Frontend:** Vue 3, TypeScript, Pinia y Vite como islas progresivas sobre vistas HTML renderizadas por PHP.
- **Persistencia:** MariaDB 10.4 mediante PDO y consultas preparadas.
- **Seguridad de archivos:** ClamAV (TCP INSTREAM); los documentos se guardan en almacenamiento privado, fuera de `public/`.
- **Infraestructura:** Docker Compose v2 con volúmenes persistentes para la base de datos, almacenamiento y firmas antivirus.
- **Documentación:** portal VitePress con diagramas Mermaid.

2. Requisitos previos

- Git.
- Docker Desktop actualizado con Docker Compose v2 (`docker compose`, no el ejecutable antiguo `docker-compose`).
- Node.js 22–24 y npm para ejecutar Vite, pruebas de tipos o VitePress directamente en el host.
- PowerShell 5.1 o posterior para los comandos de esta guía en Windows.

El contenedor `app` incluye PHP 8.2, Apache y las extensiones PDO MySQL y mbstring; no es necesario instalar PHP en Windows para el flujo Docker.

La arquitectura se organiza en tres capas: presentación (HTML/PHP y pequeñas islas Vue), aplicación (rutas, middleware, controladores y reglas PHP) y datos (MariaDB y almacenamiento privado). Vue no sustituye la navegación MVC ni la autorización del servidor.

3. Preparar el entorno

Clona el repositorio:

```
git clone https://github.com/campusedx/Practicas_Desarrollo.git
```

Entra en la carpeta recién creada:

```
Set-Location .\Practicas_Desarrollo
```

Crea `.env` con los valores locales de referencia; Compose lo carga automáticamente desde la raíz:

```
Copy-Item .env.example .env
```

Abre `.env` para cambiar puertos, seleccionar la base de datos y ajustar las credenciales locales:

```
notepad .env
```

Variables incluidas en la plantilla:

Variable	Predeterminado	Uso
`APP_PORT`	`8080`	Puerto HTTP de PHP/Apache en el host, enlazado solo a `127.0.0.1`.
`APP_ENV`	`development` en `.env.example`; `production` si Compose no recibe un valor	Entorno de ejecución; el bypass de ClamAV sólo se permite en `development`, `test` o `testing`.
`FCT_CLAMAV_ENABLED`	`false` en las plantillas locales; `true` si Compose no recibe un valor	`false` omite el escaneo sólo con `APP_ENV=development`, `test` o `testing`; usa `true` para escanear.
`PMA_PORT`	`8082`	Puerto local de phpMyAdmin; se publica sólo en `127.0.0.1`.
`FCT_DB_HOST`	`db`	Servidor MariaDB usado por la aplicación. Usa `host.docker.internal` para conectar Docker con MariaDB de XAMPP en Windows.
`FCT_DB_NAME`	`sistema_fct`	Base de datos de desarrollo.
`FCT_DB_USER`	`fct`	Usuario que utiliza la aplicación y el formulario de acceso de phpMyAdmin.
`FCT_DB_PASS`	`fct_local_dev`	Contraseña del usuario de aplicación MariaDB.
`MARIADB_ROOT_PASSWORD`	`root_local_dev`	Contraseña administrativa de MariaDB, reservada para tareas locales de administración.
`FCT_VITE_DEV_SERVER`	vacío	URL pública del servidor Vite que PHP debe incluir durante desarrollo, por ejemplo `http://localhost:5173`.
`VITE_PORT`	`5173`	Puerto de Vite cuando se ejecuta desde el perfil Docker `frontend-dev`.
`FCT_MATCH_DATE_TOLERANCE_DAYS`	`5`	Retraso máximo del inicio o adelanto máximo del fin de disponibilidad para admitir una coincidencia; entre `0` y `30` días.
`FCT_MATCH_MIN_OVERLAP_DAYS`	`1`	Días calendario coincidentes que debe cubrir como mínimo la disponibilidad; entre `1` y `366`.
`FCT_MATCH_MIN_PRACTICE_DAYS`	`1`	Días con horario de prácticas posibles que debe ofrecer como mínimo el periodo coincidente; entre `1` y `366`.
`FCT_MATCH_MIN_SCHEDULE_PERCENT`	`80`	Compatibilidad horaria mínima requerida; entre `0` y `100`.

Los valores de `.env.example` son exclusivamente para desarrollo local; sustitúyelos por claves propias y no compartas ni confirmes `.env`.

Para conservar una base de datos existente en XAMPP mientras ejecutas PHP en Docker, configura en `.env` `FCT_DB_HOST=host.docker.internal` y establece `FCT_DB_NAME`, `FCT_DB_USER` y `FCT_DB_PASS` con los valores de esa instalación. No uses `localhost` como host: dentro del contenedor se refiere al propio contenedor, no a Windows. Inicia MySQL en XAMPP y asegúrate de que el usuario tenga permisos sobre la base seleccionada. Después recrea sólo la aplicación para que tome la configuración:

```
docker compose up -d --force-recreate app
```

Este modo comparte los datos de XAMPP y no importa ni borra su base de datos. El servicio MariaDB de Docker puede seguir ejecutándose, pero la aplicación usa el host configurado en `FCT_DB_HOST`.

Compose crea automáticamente el volumen privado `fct_storage` y lo monta como `/var/lib/fct/storage`, que es el valor de `FCT_STORAGE_PATH` dentro del contenedor. El contenedor verifica y corrige al arrancar la propiedad y los permisos de los directorios de convenios, incluso si el volumen ya existía; no se debe mover a `public/`.

Si ejecutas PHP directamente con XAMPP en vez de Docker, configura `FCT_STORAGE_PATH` como una ruta absoluta a un directorio existente, con permisos de escritura para Apache y separado del document root. La aplicación rechaza rutas inexistentes o dentro de `public/`.

Puesta en producción

Esta guía sirve para entregar y preparar el sistema en una organización; no convierte el `docker-compose.yml` de desarrollo en una configuración de producción certificada. El repositorio no incluye un archivo Compose de producción listo para desplegar. El Compose disponible monta el código del host, inicia servicios de desarrollo como Mailpit y phpMyAdmin y publica puertos locales. Prepara y revisa una configuración de producción específica antes de exponer la aplicación.

1. Definir alojamiento, acceso y requisitos de firma

- Decide quién administra el sistema, quién puede firmar y qué nivel de firma exige la organización. Una firma PAdES técnicamente válida no es automáticamente una firma cualificada eIDAS. Si se requiere ese nivel, define la política y el prestador cualificado con asesoramiento jurídico antes de desplegar.
- Usa un servidor mantenido, un dominio propio, HTTPS válido y un proxy inverso/firewall configurados para publicar únicamente HTTPS. Restringe el acceso administrativo y aplica las políticas de retención y protección de datos de la organización.
- Usa una versión revisada e identificable del código e imagen; valida primero el despliegue y las migraciones en preproducción con una copia protegida de los datos. No publiques una rama de trabajo directamente.

2. Endurecer el despliegue

Construye una imagen inmutable a partir de la versión elegida y usa una configuración Compose de producción revisada. No montes el repositorio del host sobre `/var/www/html` ni uses los volúmenes locales de `vendor` como sustituto de la imagen. Excluye del despliegue público `frontend-dev`, phpMyAdmin, Mailpit y los servicios de pruebas; el Compose actual hace que `app` dependa de Mailpit, por lo que la configuración de producción debe retirar también esa dependencia. No expongas MariaDB ni ClamAV a Internet. Mantén ClamAV en una red privada y habilitado.

El puerto de `app` en el Compose actual está enlazado a `127.0.0.1`; sirve detrás de un proxy inverso en el mismo host. Si el proxy está en otro contenedor o servidor, configura una red privada y un enlace de servicio apropiados, sin publicar el puerto de aplicación directamente a Internet. Conserva únicamente el acceso HTTPS público y limita los puertos de administración a una VPN o red autorizada.

3. Inyectar configuración y secretos de producción

No reutilices `.env.example`: contiene credenciales locales, HTTP, Mailpit y una configuración mock. Inyecta los valores desde el gestor de secretos de la plataforma. Si el despliegue usa un archivo `.env`, créalo en el servidor con acceso restringido, exclúyelo del repositorio y de las copias no cifradas, y no lo pegues en incidencias, registros o comandos compartidos.

Configura como mínimo:

Variable	Configuración de producción
`APP_ENV`	`production`.
`FCT_APP_URL`	URL canónica pública, por ejemplo `https://fct.ejemplo.es`, sin barra final.
`FCT_CONVENIO_SIGNATURE_MODE`	`autofirma`. Es necesario indicarlo explícitamente; el modo predeterminado en producción es `disabled`. `mock` y `test` se rechazan fuera de desarrollo/pruebas.
`FCT_CLAMAV_ENABLED`	`true`. No habilites el bypass de análisis en producción.
`FCT_VITE_DEV_SERVER`	Vacío; sirve los assets compilados de la imagen, no Vite HMR.
`FCT_DB_NAME`, `FCT_DB_USER`, `FCT_DB_PASS`	Base y usuario exclusivos de la aplicación, con una contraseña aleatoria y permisos mínimos sobre esa base.
`MARIADB_ROOT_PASSWORD`	Secreto administrativo distinto del usuario de aplicación; no lo uses desde la aplicación.
`FCT_MAIL_TRANSPORT`, `FCT_MAIL_HOST`, `FCT_MAIL_PORT`, `FCT_MAIL_ENCRYPTION`	SMTP real de la organización (`smtp`, puerto 587 con `tls` o 465 con `ssl`), nunca Mailpit.
`FCT_MAIL_USERNAME`, `FCT_MAIL_PASSWORD`, `FCT_MAIL_FROM`	Credenciales SMTP y remitente autorizado por el proveedor de correo. Comprueba también la entregabilidad del dominio.
`FCT_AUTOFIRMA_NSS_PATH`	Ruta del servidor que contiene la base NSS de confianza preparada para la verificación PAdES; se monta como solo lectura en el contenedor.
`FCT_INTEGRATIONS_ENABLED`	`false`, salvo que se haya aprobado y configurado expresamente el uso de la API de integraciones.
`FCT_WEBHOOK_ENCRYPTION_KEY`	Clave aleatoria generada y custodiada por la organización si se habilitan webhooks; no dejes el valor vacío ni reutilices claves entre entornos.

Deja en blanco `FCT_CONVENIO_MOCK_P12_PASSWORD` y no copies al servidor el PKCS#12, la clave privada ni el NSS de desarrollo que genera `scripts/setup-autofirma-dev.js`. La ruta de `FCT_STORAGE_PATH` en la imagen es `/var/lib/fct/storage`; no la cambies por `public/`.

4. Preparar base de datos, migraciones y almacenamiento

Usa una base dedicada, con acceso de red privado y credenciales distintas por entorno. Antes de migrar una base existente, realiza y comprueba una copia de seguridad recuperable. Las migraciones automáticas del Compose de desarrollo sólo se ejecutan al inicializar un volumen MariaDB vacío; no se vuelven a aplicar al arrancar sobre una base existente.

En una base existente, identifica primero qué migraciones ya se ejecutaron y aplica sólo las pendientes, en el orden documentado en esta guía. Para la firma de convenios, `026_firma_electronica_convenios.sql` debe preceder a `027_convenio_firma_concurrencia_auditoria.sql`. **La 027 vuelve a poner en revisión convenios aprobados heredados cuya firma no acredita una PAdES verificable**; revisa ese efecto con la organización y valida el resultado en preproducción antes de ejecutarla. No importes de nuevo `database/sistema_fct.sql` ni ejecutes todas las migraciones indiscriminadamente. Consulta Base de datos, cuentas y migraciones y la guía de firma de convenios.

Monta `FCT_STORAGE_PATH` como almacenamiento persistente, privado y con permisos mínimos. Incluye los PDF originales y firmados en copias cifradas, restringe quién puede restaurarlas y prueba periódicamente una restauración. Mantén también las copias de seguridad de la base de datos coordinadas con el almacenamiento para poder recuperar metadatos y documentos en conjunto. Nunca publiques ni sirvas esos archivos desde `public/`.

5. Preparar la firma PAdES de producción

La firma nativa se realiza en el equipo del Administrador, no con una clave privada instalada en el servidor:

1. Instala y mantén AutoFirma oficial en cada equipo autorizado para firmar. Comprueba que el navegador permitido puede abrir el protocolo `afirma://` y que el equipo tiene acceso al certificado del firmante (almacén del sistema, tarjeta o dispositivo correspondiente).
2. Usa un certificado vigente y apropiado para la identidad y facultades del firmante. Define quién lo emite y custodia, cómo se renueva o revoca y qué hacer si se pierde o queda comprometido. No copies su clave privada al servidor.
3. En el servidor, prepara mediante el procedimiento de la organización una base NSS SQL en la ruta indicada por `FCT_AUTOFIRMA_NSS_PATH`. Añade únicamente las autoridades raíz e intermedias que la política haya aprobado como confiables. No importes el certificado autofirmado del modo mock ni marques certificados como confiables para hacer pasar una prueba.
4. Monta esa base en `/var/lib/fct/autofirma-nssdb` con lectura para el proceso de PHP y solo lectura desde el contenedor. El verificador `pdfsig` exige que la firma sea íntegra, que el certificado resulte confiable y que el documento completo esté firmado; además consulta OCSP. Permite desde el servidor la salida de red estrictamente necesaria para AIA/OCSP y supervisa su disponibilidad. Una incidencia de confianza, revocación o conectividad debe impedir la aprobación, no relajarse.
5. Prueba el flujo en preproducción con el certificado y la política reales del entorno. Comprueba la salida de validación PAdES, la descarga del PDF, sus hashes y el registro en `convenio_audit_logs`. El modo `test` omite OCSP y el `mock` usa una identidad local sin validez contractual: ninguno es una prueba de aceptación para producción.

La instalación de AutoFirma y el certificado del firmante son requisitos de cada equipo cliente; el almacén NSS y el acceso de validación OCSP/AIA son requisitos del servidor. Ambos lados deben estar preparados. Revisa la guía de firma de convenios para el comportamiento y las limitaciones de la implementación.

6. Puesta en servicio y operación

Antes de habilitar usuarios, verifica en el entorno de producción que la URL redirige/protege correctamente mediante HTTPS, que el inicio de sesión y los permisos funcionan, que ClamAV está saludable y que una carga se rechaza si el escáner no está disponible. Prueba el correo SMTP de extremo a extremo y confirma que no llega a Mailpit. Completa una firma PAdES controlada con un Administrador autorizado, valida el documento descargado con un lector de PDF compatible y confirma que el destinatario autorizado puede descargarlo y que otros usuarios no pueden hacerlo.

Supervisa salud y registros de PHP/Apache, MariaDB, ClamAV, SMTP y espacio/errores del almacenamiento. Establece responsables para parches de sistema e imágenes, rotación de secretos, renovación/revocación de certificados, renovación de CA confiables, copias de seguridad y respuesta ante incidentes. Antes de cada actualización, repite la validación en preproducción y prepara una recuperación probada; no uses `docker compose down -v` en producción.

Identidad visual del portal

En **Configuración > Diseño**, un Administrador puede cambiar en vivo los colores primario, secundario y de acento y subir un logo SVG, PNG o JPG de hasta 2 MB. Los colores de texto se ajustan por tema claro/oscuro para cumplir WCAG AA (4,5:1) y la vista previa conserva la proporción. Las imágenes raster se normalizan a un máximo de 512 × 128 px; los SVG se limitan a elementos vectoriales seguros. Los logos se analizan con ClamAV y permanecen en el almacenamiento privado, fuera de `public/`.

La migración `025_marca_portal.sql` se ejecuta al crear una base Docker nueva. En una base ya inicializada, aplícala una sola vez antes de abrir cualquier página de la aplicación:

```
docker compose cp (Resolve-Path .\database\migrations\025_marca_portal.sql).Path db:/tmp/migracion.sql
'mariadb --user="$MARIADB_USER" -p"$MARIADB_PASSWORD" "$MARIADB_DATABASE" < /tmp/migracion.sql' | docker compose exec
        -T db sh -s
```

Las ofertas requieren una sede de trabajo activa de su empresa. La migración `027_ofertas_sede.sql` asigna a las ofertas existentes su dirección anterior como sede, completa las ofertas legadas sin dirección con una sede principal y añade la relación obligatoria. Compose la ejecuta al inicializar una base nueva; aplícala una vez en bases existentes antes de crear ofertas:

```
Get-Content -Raw .\database\migrations\027_ofertas_sede.sql | docker compose exec -T db sh -lc 'mariadb --
        user="$MARIADB_USER" --password="$MARIADB_PASSWORD" "$MARIADB_DATABASE"'
```

4. Iniciar y detener la infraestructura

Construye la imagen de la aplicación e inicia Apache/PHP, MariaDB, ClamAV y phpMyAdmin en segundo plano:

```
docker compose up -d --build
```

Comprueba que los servicios estén activos y revisa su estado de salud:

```
docker compose ps
```

ClamAV puede tardar varios minutos en descargar sus primeras firmas. El contenedor `app` espera a que MariaDB esté saludable, pero puede iniciar mientras ClamAV prepara sus firmas; si el análisis está habilitado, una carga se rechazará hasta que el antivirus responda correctamente. Consulta su salida si todavía está inicializando:

```
docker compose logs --tail=50 clamav
```

Comprueba que el almacenamiento privado exista y sea escribible por PHP:

```
docker compose exec app sh -c 'test -d "$FCT_STORAGE_PATH" && test -w "$FCT_STORAGE_PATH"'
```

Abre la aplicación en http://localhost:8080. Para detener los contenedores sin borrar datos:

```
docker compose down
```

Los volúmenes de base de datos, archivos subidos, firmas ClamAV y dependencias Docker persisten al detener los contenedores.

5. Base de datos, cuentas de desarrollo y phpMyAdmin

En la primera inicialización de un volumen MariaDB vacío, Compose importa `database/sistema_fct.sql` y aplica las migraciones de dominio, disponibilidad, ofertas, tokens API, aprovisionamiento/activación de cuentas y trazabilidad administrativa. Los nombres de algunas migraciones comparten prefijo porque proceden de líneas de trabajo paralelas; Compose les asigna nombres de inicialización únicos para conservar el orden y evitar colisiones. La inicialización oficial de

MariaDB sólo procesa estos archivos al crear una base de datos nueva; no vuelve a ejecutarlos en cada arranque. En bases ya inicializadas, las tablas persistidas de funcionalidades retiradas no se eliminan automáticamente para proteger los datos existentes.

El alta de alumnos es aprovisionamiento interno: Administradores y Coordinadores con sesión iniciada crean las cuentas desde **Alumnado**; no hay registro público de estudiantes. El formulario exige tipo y número de documento, nombre, apellidos, correo, ciclo, curso y observaciones. Se puede elegir NIF / NIE / DNI (con verificación de letra de control) o Pasaporte, limitado a 6–15 caracteres alfanuméricos. El pasaporte se reserva para casos especiales de alumnado no residente según la normativa de Formación Profesional. El sistema genera una contraseña aleatoria que no se comparte y envía un enlace de activación de un solo uso, válido 24 horas, para que el alumno establezca su contraseña definitiva. La URL del enlace debe usar HTTPS; la única excepción es el circuito de desarrollo local con Mailpit, `APP_ENV=development` y una URL HTTP de loopback (`localhost` , `127.0.0.1` o `[::1]`). Solo se almacena el hash del token y la cuenta permanece inactiva hasta completar la activación. En una base existente, aplica una sola vez `012_alta_alumnos_personal.sql` y `013_activacion_cuentas_alumno.sql` , en ese orden, mediante el procedimiento de migración manual de abajo; no basta con reiniciar Compose.

Desde **Usuarios**, el Administrador aprovisiona cuentas institucionales de Administrador, Coordinador y Observador indicando correo, NIF / NIE / DNI con letra de control válida, nombre y apellidos. Se genera una contraseña aleatoria que no se comparte y la cuenta queda inactiva hasta que su titular use el enlace para establecer la contraseña. El enlace es un token aleatorio de 256 bits, de un solo uso y válido 24 horas; en la base sólo se guarda su hash. Su URL debe ser HTTPS, salvo el circuito local con Mailpit descrito arriba. Si el correo no se puede enviar, el alta se revierte. En una base existente, aplica una sola vez `015_alta_personal_institucional.sql` después de `014_identificador_fiscal_personal.sql` ; la migración conserva los nombres y apellidos ya registrados de Coordinadores.

El alta pública de empresas sigue el mismo flujo: no pide una contraseña inicial, crea la cuenta inactiva y envía un enlace de activación de un solo uso y válido 24 horas para elegir la contraseña definitiva. En una base existente, aplica una sola vez `019_activacion_cuentas_empresa.sql` ; en un volumen nuevo Compose lo aplica automáticamente después de las migraciones anteriores.

Si un enlace caduca o no es válido, se puede solicitar otro indicando el correo y el documento de identidad asociado a la cuenta: NIF / NIE / DNI o pasaporte del alumnado, NIF / NIE / DNI del personal, o NIF (Persona Jurídica) de empresa. La respuesta pública es genérica y no confirma si los datos corresponden a una cuenta. El control se aplica por cuenta: no se crea otra solicitud mientras haya una pendiente ni durante las 24 horas posteriores a la solicitud anterior. Un Administrador revisa la bandeja **Reenvíos de activación** y aprueba manualmente el envío; entonces se invalida el enlace anterior y se envía otro, de un solo uso y válido 24 horas. Si falla el envío, la solicitud permanece pendiente y la transacción revierte la invalidación. En una base existente, aplica una sola vez `016_solicitudes_reenvio_activacion.sql` después de las migraciones de activación `013_alta_alumnos_personal.sql` , `015_alta_personal_institucional.sql` y `019_activacion_cuentas_empresa.sql` .

Los eventos administrativos instrumentados se registran mediante `AuditLogService` en `audit_logs` y se conserva también el registro histórico en `auditoria` . La tabla incluye identificador y nombre completo del actor, acción, entidad afectada, ID del registro modificado, dirección IP, evento, detalle y fecha/hora (`timestamp`). En una base existente que aún no la tenga, aplica una sola vez `017_trazabilidad_administrativa.sql` , después de `016_solicitudes_reenvio_activacion.sql` ; reiniciar Compose no ejecuta migraciones en un volumen existente.

En una base existente, aplica una sola vez `018_tipo_documento_alumno.sql` después de las migraciones anteriores y antes de desplegar esta versión. Añade el tipo de documento al expediente del alumnado y clasifica como NIF / NIE / DNI los documentos existentes.

El acceso usa el documento de identidad y la contraseña: Alumno entra con NIF / NIE / DNI o pasaporte, el personal con NIF / NIE / DNI y Empresa con NIF (Persona Jurídica). El correo no sirve como identificador de login. El personal existente debe tener un NIF asignado en **Usuarios**; aplica una sola vez `014_identificador_fiscal_personal.sql` y conserva abierta la sesión actual del administrador hasta que este asigne allí su NIF y los del resto del personal. Si se perdió esa sesión, el administrador local puede ejecutar `docker compose exec -it app php scripts/asignar_nif_personal.php` para asignar su identificador en desarrollo; el asistente solicita correo y NIF interactivamente y no muestra contraseñas. Las cuentas antiguas sin NIF no podrán iniciar sesión hasta que se actualicen.

Para probar mensajes en local sin enviar correos reales, usa Mailpit: configura `FCT_MAIL_TRANSPORT=mailpit`, `FCT_MAIL_HOST=mailpit`, `FCT_MAIL_PORT=1025`, `FCT_MAIL_ENCRYPTION=none`, un remitente válido en `FCT_MAIL_FROM` y `FCT_APP_URL` con la URL local de la aplicación. Sin cambios locales de puertos, los valores de referencia son `http://localhost:8080` para la aplicación y http://localhost:8025 para consultar Mailpit. La aplicación acepta un enlace HTTP sólo en `APP_ENV=development` y con host loopback (`localhost`, `127.0.0.1` o `[::1]`); para cualquier otro destino el enlace debe usar HTTPS. No envíes mensajes de prueba a alumnos. Para correo real con Gmail/Google Workspace, usa `FCT_MAIL_TRANSPORT=smtp`, `FCT_MAIL_USERNAME`, `FCT_MAIL_PASSWORD` (contraseña de aplicación), `FCT_MAIL_FROM` y una URL HTTPS accesible desde los dispositivos del alumnado. Gmail requiere verificación en dos pasos; no uses la contraseña normal ni guardes credenciales en el repositorio.

Las ofertas históricas aprobadas no tienen horas exactas inferibles a partir de sus antiguos turnos amplios; se excluyen del matching y del catálogo hasta republicarlas con horario semanal. No asignes horarios aproximados automáticamente.

Para una base ya inicializada, no vuelvas a importar el esquema ni ejecutes migraciones existentes indiscriminadamente: varias no son idempotentes. Examina `docker compose ps` y los registros de MariaDB antes de diagnosticar una instalación incompleta. `009_desarrollo_aplicaciones_multiplataforma.sql` es idempotente y puede aplicarse a una base existente para añadir DAM. Para un entorno nuevo creado fuera de Compose, ejecuta el esquema y las migraciones una sola vez, en orden, contra esa base vacía.

Procedimiento para esquema y migraciones manuales. No canalices los archivos con `Get-Content -Raw … | docker compose exec …`: en Windows PowerShell 5.1, `$OutputEncoding` es US-ASCII y `Get-Content` lee como ANSI un UTF-8 sin BOM, de modo que los caracteres no ASCII llegan corruptos. Por ejemplo, `ENUM('Presencial','Remoto','Híbrido')` de `010_detalles_ofertas.sql` llegaría como `H??brido`. En su lugar, copia el archivo al contenedor `db` y ejecútalo desde allí:

```
docker compose cp (Resolve-Path .\database\sistema_fct.sql).Path db:/tmp/migracion.sql
'mariadb --user="$MARIADB_USER" --password="$MARIADB_PASSWORD" "$MARIADB_DATABASE" < /tmp/migracion.sql #' | docker
        compose exec -T db sh -s
```

Las comillas simples impiden que PowerShell expanda las variables: las resuelve `sh` dentro del contenedor. El `#` final convierte en comentario el `\r` que PowerShell 5.1 añade a la entrada estándar; sin él, ese `\r` pasa a formar parte de la ruta y `sh` responde `cannot open /tmp/migracion.sql`. `docker compose cp` escribe su progreso por stderr, y PowerShell 5.1 puede mostrarlo como `NativeCommandError`: decide el resultado con `$LASTEXITCODE`, no con ese texto. Este bloque carga el esquema inicial en la base seleccionada por las variables de Compose; úsalo sólo si el esquema aún no existe. Para aplicar todas las migraciones SQL en orden alfabético sobre esa base vacía:

```
foreach ($migration in Get-ChildItem .\database\migrations\*.sql | Sort-Object Name) {
    docker compose cp $migration.FullName db:/tmp/migracion.sql
    'mariadb --user="$MARIADB_USER" --password="$MARIADB_PASSWORD" "$MARIADB_DATABASE" < /tmp/migracion.sql #' |
        docker compose exec -T db sh -s
    if ($LASTEXITCODE -ne 0) { throw "Falló la migración $($migration.Name)" }
}
```

Cada migración se aplica una sola vez y en orden; no ejecutes este bucle sobre un volumen que ya haya recibido esas migraciones durante la inicialización de Compose.

Para una base existente que ya tiene las migraciones de dominio hasta la 006, aplica `007_horarios_ofertas.sql` y `008_disponibilidad_indiferente.sql` en orden, y sólo si no se aplicaron previamente:

```
docker compose cp (Resolve-Path .\database\migrations\007_horarios_ofertas.sql).Path db:/tmp/migracion.sql
'mariadb --user="$MARIADB_USER" --password="$MARIADB_PASSWORD" "$MARIADB_DATABASE" < /tmp/migracion.sql #' | docker
        compose exec -T db sh -s
```

Este procedimiento crea la tabla de horarios exactos de las ofertas sin volver a ejecutar migraciones antiguas; reinicia el servicio `app` después de aplicarlo para que su comprobación de salud detecte la tabla.

Para una base que ya recibió `007_horarios_ofertas.sql`, aplica únicamente el campo de disponibilidad indiferente:

```
docker compose cp (Resolve-Path .\database\migrations\008_disponibilidad_indiferente.sql).Path db:/tmp/migracion.sql
'mariadb --user="$MARIADB_USER" --password="$MARIADB_PASSWORD" "$MARIADB_DATABASE" < /tmp/migracion.sql #' | docker
        compose exec -T db sh -s
```

`008_disponibilidad_indiferente.sql` añade `es_indiferente` de forma idempotente y migra el valor anterior de `turno` sólo la primera vez; `009_desarrollo_aplicaciones_multiplataforma.sql` inserta DAM si aún no existe. En una base que ya recibió la primera, aplica la segunda para habilitar DAM:

```
docker compose cp (Resolve-Path .\database\migrations\009_desarrollo_aplicaciones_multiplataforma.sql).Path
        db:/tmp/migracion.sql
'mariadb --user="$MARIADB_USER" --password="$MARIADB_PASSWORD" "$MARIADB_DATABASE" < /tmp/migracion.sql #' | docker
        compose exec -T db sh -s
```

El matching omite la cobertura horaria diaria para el alumnado indiferente, pero sigue exigiendo fechas, ciclo, oferta aprobada con franjas y capacidad.

`010_detalles_ofertas.sql` añade título, descripción, turno, modalidad, ubicación, requisitos y tecnologías a las ofertas existentes. En una base que ya recibió `009_desarrollo_aplicaciones_multiplataforma.sql`, aplícala una vez:

```
docker compose cp (Resolve-Path .\database\migrations\010_detalles_ofertas.sql).Path db:/tmp/migracion.sql
'mariadb --user="$MARIADB_USER" --password="$MARIADB_PASSWORD" "$MARIADB_DATABASE" < /tmp/migracion.sql #' | docker
        compose exec -T db sh -s
```

La migración es idempotente. Los datos de ofertas históricas reciben valores predeterminados compatibles; una descripción/ubicación no se inventa para esos registros. Después de aplicarla, comprueba con una consulta de solo lectura que `modalidad` conserva `Híbrido` (la columna debe mostrar `enum('Presencial','Remoto','Híbrido')`):

```
'mariadb --user="$MARIADB_USER" --password="$MARIADB_PASSWORD" "$MARIADB_DATABASE" -e "SHOW COLUMNS FROM
        empresa_especialidades_plazas LIKE ''modalidad''" #' | docker compose exec -T db sh -s
```

Compose aplica `008_api_access_tokens.sql` al crear un volumen nuevo. Si conservas una base existente, aplica esa migración una sola vez con el procedimiento `docker compose cp` descrito arriba para habilitar los tokens de API.

Para habilitar el NIF del personal en una base local ya inicializada, aplica **una sola vez** la migración `014_identificador_fiscal_personal.sql`:

```
docker compose cp (Resolve-Path .\database\migrations\014_identificador_fiscal_personal.sql).Path
        db:/tmp/migracion.sql
'mariadb --user="$MARIADB_USER" --password="$MARIADB_PASSWORD" "$MARIADB_DATABASE" < /tmp/migracion.sql #' | docker
        compose exec -T db sh -s
```

No repitas este `ALTER TABLE` si `usuarios.identificador_fiscal` ya existe. Después recarga la página de la aplicación. Desde **Usuarios**, asigna y guarda el NIF/DNI/NIE válido del administrador actual y de cada miembro del personal antes de cerrar la sesión; no se puede inferir el NIF real a partir del correo.

Para habilitar el alta institucional de personal, aplica después y una sola vez `015_alta_personal_institucional.sql`:

```
docker compose cp (Resolve-Path .\database\migrations\015_alta_personal_institucional.sql).Path db:/tmp/migracion.sql
'mariadb --user="$MARIADB_USER" --****** "$MARIADB_DATABASE" < /tmp/migracion.sql #' | docker compose exec -T db sh -
        s
```

Esta migración añade nombre y apellidos para las cuentas de personal, conserva los datos ya guardados de Coordinadores y crea la tabla de enlaces de activación de personal.

Para habilitar las solicitudes y aprobación de reenvíos de activación, aplica después y una sola vez `016_solicitudes_reenvio_activacion.sql`:

```
docker compose cp (Resolve-Path .\database\migrations\016_solicitudes_reenvio_activacion.sql).Path
        db:/tmp/migracion.sql
'mariadb --user="$MARIADB_USER" --****** "$MARIADB_DATABASE" < /tmp/migracion.sql #' | docker compose exec -T db sh -
        s
```

En una base existente que todavía no tenga la tabla de trazabilidad, aplica también y una sola vez `017_trazabilidad_administrativa.sql`, después de `016_solicitudes_reenvio_activacion.sql`:

```
docker compose cp (Resolve-Path .\database\migrations\017_trazabilidad_administrativa.sql).Path db:/tmp/migracion.sql
'mariadb --user="$MARIADB_USER" --password="$MARIADB_PASSWORD" "$MARIADB_DATABASE" < /tmp/migracion.sql #' | docker
        compose exec -T db sh -s
```

Para habilitar el tipo de documento de alumnado en una base existente, aplica también y una sola vez `018_tipo_documento_alumno.sql`:

```
docker compose cp (Resolve-Path .\database\migrations\018_tipo_documento_alumno.sql).Path db:/tmp/migracion.sql
'mariadb --user="$MARIADB_USER" --****** "$MARIADB_DATABASE" < /tmp/migracion.sql #' | docker compose exec -T db sh -
        s
```

Para habilitar la activación del registro de empresas en una base existente, aplica también y una sola vez `019_activacion_cuentas_empresa.sql`:

```
docker compose cp (Resolve-Path .\database\migrations\019_activacion_cuentas_empresa.sql).Path db:/tmp/migracion.sql
'mariadb --user="$MARIADB_USER" --****** "$MARIADB_DATABASE" < /tmp/migracion.sql #' | docker compose exec -T db sh -
        s
```

Si la sesión de administrador ya no está disponible, dentro de Docker Desktop y en modo desarrollo ejecuta `docker compose exec -it app php scripts/asignar_nif_personal.php`; escribe el correo existente y el NIF real cuando el asistente los solicite. Repite para cada cuenta del personal que necesite acceso. Este asistente está limitado a ejecución CLI en `APP_ENV=development` o `local`.

Genera las cinco cuentas de prueba actuales (Administrador, Coordinador, Observador, Empresa y Alumno) sólo en la base local autorizada:

```
docker compose exec -e FCT_ENABLE_DEMO_SEED=1 -e FCT_DEMO_SEED_ALLOW_DOCKER=1 app php scripts/crear_cuentas_demo.php
```

El script imprime identificadores fiscales y contraseñas aleatorias una sola vez. **Cada ejecución renueva todas las contraseñas demo y deja inválidas las anteriores**; guarda la salida de forma segura. También crea para el alumno demo disponibilidad de lunes a viernes, de 09:00 a 17:00, con franjas exactas. No ejecutes este comando en producción. `database/seeds/001_sample_data.sql` es un conjunto legado de datos de muestra y no se ejecuta automáticamente ni sustituye al generador seguro de cuentas.

La cuenta demo de Administrador usa el correo `carmen.demo@fct.local` (el esquema rechaza correos genéricos como `admin@` para Administradores y exige nombre y apellidos) y el rol `Administrador`, pero el formulario de inicio de sesión no admite correo: entra con el identificador fiscal y la contraseña que aparecen en la salida de esa ejecución. El identificador

cambia en cada ejecución del script. Las contraseñas no se guardan en texto legible ni se pueden consultar después. Si perdiste la salida, puedes volver a ejecutar el comando únicamente en desarrollo local; generará credenciales nuevas y cambiará las contraseñas de **todas** las cuentas demo, así que guarda de forma segura la nueva salida.

Consulta la matriz de roles y permisos para conocer el acceso restringido al catálogo y el alcance de solo lectura del perfil Observador.

Abre phpMyAdmin en http://localhost:8082. Inicia sesión con el servidor `db`, puerto `3306`, base `sistema_fct` y las credenciales `FCT_DB_USER` / `FCT_DB_PASS` de `.env` (por defecto, `fct`). El usuario `root` y `MARIADB_ROOT_PASSWORD` deben reservarse para administración local cuando sea necesaria; no uses phpMyAdmin para exponer el servicio a la red.

El puerto MariaDB publicado para clientes del host es `127.0.0.1:3307`; dentro de Compose, la aplicación y phpMyAdmin se conectan al nombre de servicio `db:3306`.

Importación de empresas desde Excel o PDF

Administrador y Coordinador pueden acceder a `index.php?action=importar-empresas`. El Administrador también dispone del acceso **Empresas → Gestión de empresas → Importar empresas**. Se admiten archivos `.xlsx` o `.pdf` de hasta **5 MB y 100 empresas**.

- Excel: columnas **Razón social, NIF, Email y Representante**; las cabeceras permiten identificar las columnas aunque cambien de orden.
- PDF normalizado: texto seleccionable, una empresa por línea con el formato `Razón social; NIF; Email; Representante`. No se realiza OCR de documentos escaneados como imagen.
- Cada fila valida el NIF jurídico, correo y campos obligatorios. Los duplicados de NIF o correo se notifican sin impedir el procesamiento de las demás filas. Las filas PDF incompletas también aparecen en el informe de errores.
- Cada alta crea una cuenta Empresa inactiva y un token aleatorio de 256 bits, de un solo uso y válido 24 horas. Solo se guarda su hash en `company_activation_tokens`, reutilizando el esquema de `019_activacion_cuentas_empresa.sql`. Si el envío del correo falla, se revierte el alta de esa fila.
- `AuditLogService` registra cada empresa creada y el resumen de la importación en `audit_logs`, incluyendo cantidades procesadas, correctas y fallidas y el hash del archivo.

PHP recibe la carga en su directorio temporal. Antes de extraer datos, la aplicación analiza ese temporal mediante **ClamAV TCP INSTREAM**. No conserva una copia permanente del archivo importado. Con el análisis habilitado, una detección, un fallo del servicio o firmas con más de tres días bloquean el procesamiento.

Si el entorno existente no encuentra las clases de Excel o PDF, instala las dependencias declaradas en el archivo de bloqueo:

```
docker compose exec -T app composer install --no-interaction --prefer-dist --no-dev
```

En desarrollo, con el transporte Mailpit configurado, consulta los correos de activación y aprobación de convenios en http://localhost:8025. Estos mensajes quedan en el buzón local; no se entregan a los destinatarios externos. La excepción de activación por HTTP requiere `APP_ENV=development`, `FCT_APP_URL=http://localhost:8080`, `FCT_MAIL_TRANSPORT=mailpit`, `FCT_MAIL_HOST=mailpit`, `FCT_MAIL_PORT=1025`, `FCT_MAIL_ENCRYPTION=none` y un `FCT_MAIL_FROM` válido. Tras modificar `.env`, recrea `app` para cargar los valores nuevos.

Habilitar el archivo de convenios en una base existente

La vista unificada `index.php?action=gestionar-convenios` reúne pendientes, aprobados, rechazados y archivados, con filtros por NIF, estado y fecha de registro, y paginación. La aprobación solo se completa tras validar una firma criptográfica PAdES. El servidor conserva el original y el PDF firmado en almacenamiento privado, registra los hashes SHA-256 y envía el PDF

firmado como adjunto a la empresa. La confianza de un certificado depende del almacén configurado; una firma válida no implica por sí sola que sea cualificada conforme a eIDAS.

En instalaciones existentes aplica `024_convenios_estado_archivado.sql` para añadir `Archivado` sin eliminar los estados ni los datos anteriores. Reiniciar los servicios no aplica migraciones a un volumen ya inicializado:

```
docker compose cp (Resolve-Path .\database\migrations\024_convenios_estado_archivado.sql).Path db:/tmp/migracion.sql
'mariadb --user="$MARIADB_USER" --password="$MARIADB_PASSWORD" "$MARIADB_DATABASE" < /tmp/migracion.sql #' | docker
        compose exec -T db sh -s
```

Después de migraciones hasta la 025, aplica `026_firma_electronica_convenios.sql` para los metadatos de firma y `027_convenio_firma_concurrencia_auditoria.sql` para el estado, la reserva temporal de firma y su auditoría:

```
docker compose cp (Resolve-Path .\database\migrations\026_firma_electronica_convenios.sql).Path db:/tmp/migracion.sql
'mariadb --user="$MARIADB_USER" --****** "$MARIADB_DATABASE" < /tmp/migracion.sql #' | docker compose exec -T db sh -
        s
docker compose cp (Resolve-Path .\database\migrations\027_convenio_firma_concurrencia_auditoria.sql).Path
        db:/tmp/migracion.sql
'mariadb --user="$MARIADB_USER" --****** "$MARIADB_DATABASE" < /tmp/migracion.sql #' | docker compose exec -T db sh -
        s
```

Las migraciones 026/027 añaden metadatos y, en el caso de la 027, reabren aprobaciones heredadas para revalidar su firma. No reinicialices una base de datos existente para aplicarlas.

Al aplicar la migración 027, las aprobaciones antiguas con un PDF asociado pero sin estado `FIRMADO_Y_APROBADO` vuelven a `Pendiente`. Se conserva su hash y nombre de archivo anterior en `convenio_audit_logs`; el archivo legado permanece en almacenamiento privado y deberá reemplazarse con una firma PAdES verificada.

Para crear un certificado local de prueba, un almacén NSS y un PDF ficticio, ejecuta `node scripts/setup-autofirma-dev.js` con Docker Compose iniciado. El script genera una contraseña aleatoria y actualiza las dos variables del modo mock en `.env`; después recrea `app` con `docker compose up -d --force-recreate app`. El modo mock solo está disponible en desarrollo/pruebas y su certificado no tiene validez contractual. Para usar AutoFirma real configura `FCT_CONVENIO_SIGNATURE_MODE=autofirma` y el almacén NSS de la organización.

6. Recorrido funcional: disponibilidad, matching y seguimiento

El sistema compara ofertas y expedientes del mismo ciclo formativo, por ejemplo DAW con DAW y DAM con DAM. Para que una asignación sea válida, la oferta debe estar aprobada, tener franjas semanales publicadas y plazas disponibles. El criterio temporal admite por defecto hasta cinco días de retraso en el inicio o de adelanto en el fin, siempre que quede al menos un día calendario coincidente; esos umbrales se configuran con `FCT_MATCH_DATE_TOLERANCE_DAYS` y `FCT_MATCH_MIN_OVERLAP_DAYS`. Además, el periodo coincidente debe ofrecer al menos un día de prácticas con horario (`FCT_MATCH_MIN_PRACTICE_DAYS`) y conservar las horas mínimas necesarias. El periodo asignado se adapta al tramo que realmente coincide. La pantalla de emparejamiento muestra los umbrales activos, y el servidor vuelve a validar todos los criterios al asignar.

Los ciclos soportados incluyen **DAW** (Desarrollo de Aplicaciones Web) y **DAM** (Desarrollo de Aplicaciones Multiplataforma). La especialidad se relaciona por ID en la base de datos, y `009_desarrollo_aplicaciones_multiplataforma.sql` incorpora DAM sin duplicarlo. Una oferta guarda un único ciclo para preservar el matching actual.

La publicación recoge título, descripción, plazas, fechas, turno, franjas semanales de cinco minutos, modalidad, ubicación, requisitos y tecnologías. La modalidad presencial e híbrida requiere ubicación. La revisión y los listados muestran los detalles persistidos; el matching sigue usando la misma relación por ciclo, fechas y horarios.

En el portal del alumno, la disponibilidad tiene tres pasos de configuración:

1. Indica las fechas de inicio y fin del periodo.

2. Elige una de estas opciones horarias:
    - **Franjas exactas:** marca cada día y define su hora de inicio y fin. Puedes aplicar un horario general a varios días y luego editar cada día.
    - **Disponibilidad indiferente:** indica que aceptas cualquier horario de oferta; las franjas por día quedan desactivadas. Fechas y ciclo siguen siendo obligatorios.
3. Envía el formulario para guardar. La isla Vue muestra de forma reactiva el estado de guardado o el error; no guarda cambios automáticamente. Sin JavaScript, el formulario HTML/PHP mantiene el envío y la validación del servidor.

Con franjas exactas, cada intervalo ofertado `[inicio, fin]` debe quedar contenido en un intervalo de disponibilidad del mismo día (`disponibilidad.inicio <= oferta.inicio` y `disponibilidad.fin >= oferta.fin`). La opción indiferente omite únicamente esa comparación horaria: no omite el ciclo, las fechas, la aprobación, las franjas publicadas ni el control de plazas. Las ofertas históricas sin horarios semanales exactos no participan en el matching ni se pueden aprobar hasta volver a publicarse o hasta que un Administrador registre el horario acordado desde la revisión de ofertas; este cambio queda auditado.

Administrador y Coordinador registran calificaciones de prácticas cerradas; se admiten notas de **0.00 a 10.00**, con hasta dos decimales y sin caracteres alfanuméricos. El alumno consulta sus propias notas. El tema claro/oscuro se administra mediante Pinia, conserva la elección en `localStorage` y ofrece un selector visual SVG. El perfil Observador es de solo lectura y recibe datos anonimizados, sin acceso a documentos privados. La política completa está en Roles y permisos.

Rol	Permisos principales
Administrador	Gestión de usuarios, ofertas/convenios y decisiones de revisión; asignaciones y calificaciones.
Coordinador	Seguimiento, asignaciones, cierre de prácticas, calificaciones y consultas; no aprueba ofertas o convenios mediante las rutas MVC.
Observador	Sólo lectura, con datos anonimizados y sin documentos privados; `POST /logout` sólo invalida su sesión.
Alumno	Su perfil, disponibilidad, asignaciones y calificaciones, más información de su empresa asignada.
Empresa	Sus convenios, sedes y ofertas; puede descargar desde su cuenta los convenios aprobados y firmados digitalmente. No decide sus revisiones.
Invitado	Inicio, registro, autenticación y recuperación; las áreas protegidas requieren sesión.

La revisión y el histórico de convenios se consultan desde una vista unificada (`index.php?action=gestionar-convenios`) con pestañas para pendientes, aprobados, rechazados y archivados, filtros por NIF y fecha de registro, y paginación. `revisar-convenios` y `archivo-convenios` siguen disponibles como accesos compatibles a esa vista. Solo Administrador puede aprobar, rechazar o archivar. La aprobación requiere una firma PAdES validada, registra hashes y metadatos en la auditoría específica de convenios y adjunta el PDF firmado al correo de la empresa. La empresa solo descarga copias de sus propios convenios aprobados. El original y el PDF firmado se conservan en almacenamiento privado fuera de `public/` .

El botón de tema claro/oscuro sigue la preferencia del sistema si todavía no hay una elección guardada; al cambiarlo manualmente, la elección queda en el almacenamiento local del navegador.

La matriz anterior describe la navegación MVC protegida por `routes/web.php` . `public/api.php` es una API tokenizada separada y aplica su propia matriz; en la implementación actual, las revisiones de ofertas y convenios también están reservadas a Administrador. Comparte el sobre JSON de éxito/error de `JsonResponse` : éxito `{ "success": true, "data": T }` y error `{ "success": false, "error": "..." }` . En las listas, la paginación va en `data.items` y `data.meta` ; los códigos de error se envían en `X-API-Error-Code` . Consulta API REST antes de emitir tokens o integrar clientes; las vistas anonimizadas de Observador del MVC no equivalen a la salida REST.

Antivirus en desarrollo local

Compose ejecuta ClamAV y PHP se conecta a él por la red interna del proyecto. Si no se definen variables de entorno, Compose usa `APP_ENV=production` y `FCT_CLAMAV_ENABLED=true`; las plantillas locales pueden reemplazarlos explícitamente. Para pruebas locales sin antivirus, define ambos `APP_ENV=development` y `FCT_CLAMAV_ENABLED=false`; el modo de bypass registra una advertencia y también se admite en los entornos `test` y `testing`. Para probar el escaneo real, establece `FCT_CLAMAV_ENABLED=true` y espera a que `docker compose ps` muestre `clamav` saludable. En producción, deja el escaneo habilitado: configuración inválida, fallo de conexión, firmas caducadas, timeout o detección de amenaza rechazan la carga.

Si necesitas comprobar el comportamiento de fallo cerrado, establece `FCT_CLAMAV_ENABLED=true`, detén ClamAV sólo en un entorno local desechable (`docker compose stop clamav`), confirma que una carga se rechaza y vuelve a iniciarlo (`docker compose up -d clamav`). No desactives el análisis fuera de desarrollo ni expongas el puerto interno del scanner.

7. Acceso a la aplicación y herramientas

Consulta la guía de acceso para usuarios o su fuente editable para iniciar el entorno y acceder según el perfil.

Servicio	Dirección local
Aplicación PHP	http://localhost:8080 (o `APP_PORT`)
phpMyAdmin	http://localhost:8082 (o `PMA_PORT`)
Vista previa de disponibilidad (sólo desarrollo)	http://localhost:8080/index.php?action=preview-disponibilidad
Vista previa de publicar oferta (sólo desarrollo)	`http://localhost:8080/dev/preview-publicar-oferta`
Vite HMR	http://localhost:5173 cuando se inicia el frontend
Portal VitePress	http://localhost:5174 con el comando de desarrollo de documentación

Estas direcciones reflejan los puertos predeterminados de `docker-compose.yml` y `.env.example`. Si el entorno usa `docker-compose.override.yml`, sus mapeos locales pueden sustituirlos; consulta ese archivo y ajusta `FCT_APP_URL` a la dirección local efectiva. Los valores del override no son los predeterminados compartidos.

El acceso a páginas y acciones dentro de la aplicación continúa gobernado por `routes/web.php` y el rol de la sesión; conocer la URL no concede permisos.

La página **Integraciones**, visible sólo para Administrador, ofrece una consola para probar la API REST propia.

8. Frontend y Vite HMR

Instala las dependencias fijadas en `package-lock.json` para trabajar con Vite y Vue desde el host:

```
npm ci
```

Define la URL del servidor Vite en `.env` para que las vistas PHP incluyan el cliente HMR y los módulos frontend durante desarrollo:

```
FCT_VITE_DEV_SERVER=http://localhost:5173
```

Aplica el cambio de entorno recreando el servicio PHP si ya estaba iniciado:

```
docker compose up -d app
```

Inicia Vite en el puerto usado en la URL anterior:

```
npm run dev
```

Vite sirve HMR en http://localhost:5173. Si el puerto está ocupado, libera el puerto o cambia `VITE_PORT` (perfil Docker) y `FCT_VITE_DEV_SERVER` para que la URL configurada coincida con la URL accesible desde el navegador.

También puedes iniciar el servidor Vite en un contenedor con el perfil opcional:

```
docker compose --profile frontend-dev up -d
```

El perfil instala dependencias con `npm ci`, ejecuta Vite escuchando en todas las interfaces del contenedor y publica `VITE_PORT` (5173 por defecto). Para regresar al bundle de producción incluido en la imagen, detén el perfil y vacía `FCT_VITE_DEV_SERVER` en `.env`.

Las vistas siguen renderizándose en PHP y conservan sus formularios como fallback; Vue se monta sólo en elementos marcados `data-vue-island`. Las acciones JSON del MVC usan el contrato `{ success, data }` / `{ success, error }`, envían `X-CSRF-Token` y no reemplazan la navegación MVC por rutas SPA. La API tokenizada de `public/api.php` conserva ese sobre discriminado y añade `items` / `meta` en las listas; consulta API REST.

CSS compartido y PostCSS

`frontend/src/main.ts` importa la hoja activa `public/assets/css/style.css`, por lo que Vite aplica el PostCSS configurado en `postcss.config.js` (`postcss-nested` y Autoprefixer; `vite.config.ts` lo referencia en `css.postcss`) y publica el CSS junto al bundle de Vue. En la imagen Docker, la etapa `frontend-build` del `Dockerfile` copia `postcss.config.js` y `public/assets/css` (además de `tsconfig.json`, `vite.config.ts` y `frontend`) antes de ejecutar `npm run build`; sin esa configuración, Vite omite PostCSS sin error y el bundle se generaría sin reglas anidadas resueltas ni prefijos, por lo que conviene que ambos archivos sigan en esa etapa. El layout PHP carga el CSS compilado desde el manifiesto de Vite en producción. En desarrollo (`FCT_VITE_DEV_SERVER` definido) Vite inyecta el CSS mediante JavaScript, que se ejecuta de forma diferida; para evitar un parpadeo sin estilos al cambiar de pantalla, `app/Views/layouts/main.php` enlaza además `assets/css/style.css` de forma bloqueante en el `<head>`. Si no hay bundle ni servidor HMR, ese mismo enlace actúa como fallback. Vite ya minifica los assets de producción; no se añade Tailwind ni CSSnano porque el proyecto no usa esas herramientas.

Verifica ambos lados del pipeline después de tocar los estilos:

```
npm run typecheck
npm run build
npm run docs:build
```

Cuando los cambios de una isla Vue no aparecen

La isla de disponibilidad actual se llama `AvailabilitySchedule` (archivo `frontend/src/islands/AvailabilitySchedule.vue`); el registro de `frontend/src/main.ts` descubre automáticamente los componentes Vue y la vista PHP debe usar el mismo valor en `data-vue-island`.

- **Desarrollo con HMR:** define `FCT_VITE_DEV_SERVER=http://localhost:5173` en `.env`, recrea PHP con `docker compose up -d app` para cargar esa variable, y ejecuta `npm run dev` en el host (o inicia el perfil `frontend-dev`). Comprueba que el navegador puede abrir `http://localhost:5173`.
- **Bundle de producción Docker:** `npm run build` genera `public/build` en el host, pero no actualiza una imagen PHP ya creada. Reconstruye y recrea la aplicación con `docker compose up -d --build app`; la imagen copia el bundle durante el build.
- **Comprueba el montaje:** abre la consola y la pestaña Network del navegador, fuerza recarga sin caché y confirma que se solicita el entrypoint de Vite o el bundle `build/assets/main-*.js`. Revisa que el HTML entregue el marcador `data-vue-island` y que su nombre corresponda a un archivo en `frontend/src/islands/`.
- **Si HMR conserva una versión antigua:** compara el módulo servido en `http://localhost:5173/frontend/src/islands/AvailabilitySchedule.vue` con el archivo del host. Si no coinciden, reinicia `docker compose restart frontend-dev` (o vuelve a iniciar el proceso `npm run dev`) para limpiar las transformaciones en memoria de Vite; después recarga el navegador sin caché. `docker compose down -v` no es necesario para invalidar HMR.
- **Aísla la vista:** `.env.example` establece `APP_ENV=development` para el entorno local; el valor de reserva de Compose es `production`. Las vistas previas (`preview-disponibilidad` y `preview-publicar-oferta`) sólo se registran cuando `APP_ENV` es `development` o `local` (`DevelopmentPreviewController::isEnabled()`). Usa `http://localhost:8080/index.php?action=preview-disponibilidad` para mostrar la isla con datos de prueba sin iniciar sesión; configura `APP_ENV=production` en despliegues para que las rutas no se registren. El bypass de ClamAV tiene un criterio distinto: sólo vale para `development`, `test` o `testing`, no para `local`.

Valida el frontend tras modificar componentes con `npm run typecheck` y `npm run build`. No hace falta purgar volúmenes para desplegar cambios de interfaz: basta con HMR o reconstruir la imagen de la aplicación.

9. Portal de documentación VitePress

Usa `npm ci` desde la raíz si aún no instalaste las dependencias Node. Para evitar competir con Vite HMR en el puerto 5173, ejecuta la documentación en el puerto 5174:

```
npm run docs:dev -- --port 5174
```

Abre http://localhost:5174; si el puerto indicado también está ocupado, VitePress asignará otro puerto disponible y lo mostrará en la terminal. El portal está en `docs_md/` y renderiza los diagramas Mermaid.

Compila el portal para verificar páginas Markdown, configuración y diagramas:

```
npm run docs:build
```

Sirve localmente la última compilación de VitePress:

```
npm run docs:preview -- --port 4173
```

10. Verificaciones y pruebas

Comprueba los tipos de los componentes e islas Vue/TypeScript:

```
npm run typecheck
```

Genera el bundle de producción y ejecuta de nuevo la comprobación de tipos:

```
npm run build
```

Ejecuta las pruebas de seguridad de almacenamiento y análisis de archivos en el contenedor PHP (las pruebas usan un directorio temporal y no requieren la base de datos):

```
$tests = (Resolve-Path .\tests).Path -replace '\\', '/'
docker compose run --rm --no-deps -v "${tests}:/var/www/html/tests:ro" app php tests/file_security.php
```

Las pruebas de documentos pueden ejecutarse sin base de datos con `docker compose exec -T app php tests/identity_documents.php`. Cubren la letra de control de DNI/NIE bajo la etiqueta visible **NIF / NIE / DNI**, el formato de pasaporte (6–15 caracteres alfanuméricos), el cambio de validación según el tipo seleccionado, la nota informativa accesible, el formulario de empresa sin contraseña inicial y el rótulo **NIF (Persona Jurídica)**. También se ejecutan desde `tests/file_security.php`.

La suite de integración con base de datos se ejecuta en una instancia MariaDB aislada, con almacenamiento temporal y sin puerto publicado. Inicializa el esquema, todas las migraciones y los datos de ejemplo; al finalizar elimina el contenedor y sus datos. Desde PowerShell puedes repetirla tantas veces como sea necesario:

```
.\scripts\run_integration_tests.ps1
```

El servicio aislado usa la base `fct_test_integration`; no modifica `sistema_fct` ni necesita cambiar sus credenciales o permisos. `tests/run.php` y las demás pruebas DB rechazan cualquier base que no use el prefijo `fct_test_`.

Para comprobar la extracción de Excel y PDF normalizado, las filas incompletas y el rechazo de archivos corruptos, ejecuta:

```
docker compose exec -T app php tests/company_import.php
```

Las pruebas de importación con base de datos verifican cuentas inactivas, duplicados, errores por fila, caducidad de tokens y auditoría. Las de convenios comprueban filtros, transiciones y permisos. La suite nueva ejecuta estos escenarios además de la integración principal, el horario histórico de ofertas y el matching temporal. La prueba de importación envía mensajes de prueba a Mailpit si se configura el transporte; el runner aislado usa el entorno de desarrollo y no requiere credenciales SMTP.

Restablecimiento de Base de Datos por Cambios de Esquema o Migraciones

> ⚠️ **ADVERTENCIA: restablecimiento destructivo.** `docker compose down -v` elimina todos los volúmenes nombrados de este proyecto Compose, no sólo el de MariaDB. Se perderán la base de datos (`db_data`), los archivos privados subidos (`fct_storage`), las firmas de ClamAV (`clamav_db`) y las dependencias Docker (`node_modules` y `frontend_node_modules`). Es irreversible; usa este procedimiento sólo en un entorno local desechable y después de guardar cualquier dato que quieras conservar.

Alternativa más segura si sólo necesitas reinicializar la base de datos: detén el proyecto, elimina únicamente el volumen `db_data` y vuelve a iniciarlo. Así se conservan el almacenamiento privado, ClamAV y las dependencias:

```
docker compose down
$dbVolume = (docker compose config --format json | ConvertFrom-Json).volumes.db_data.name
docker volume rm $dbVolume
docker compose up -d --build
```

Para restablecer **todos** los datos locales después de añadir cambios de esquema o migraciones, sigue estos tres pasos:

1. Elimina los contenedores y todos los volúmenes del proyecto:

   ```
   docker compose down -v
   ```

2. Reconstruye e inicia los servicios. MariaDB importará el esquema y ejecutará las migraciones configuradas al crear su volumen vacío:

   ```
   docker compose up -d --build
   ```

3. Si necesitas las cuentas demo, créalas de nuevo en la base local:

   ```
   docker compose exec -e FCT_ENABLE_DEMO_SEED=1 -e FCT_DEMO_SEED_ALLOW_DOCKER=1 app php
           scripts/crear_cuentas_demo.php
   ```

   El script renueva las contraseñas demo en cada ejecución; guarda la salida de forma segura. No lo ejecutes en producción.

11. Solución de problemas frecuentes

Puertos ocupados

Los puertos predeterminados son `8080` (aplicación), `8082` (phpMyAdmin), `3307` (MariaDB del host) y `5173` (Vite HMR). Cambia `APP_PORT`, `PMA_PORT` o `VITE_PORT` en `.env` para los puertos publicados configurables; el puerto de MariaDB se cambia editando el mapeo explícito `127.0.0.1:3307:3306` en `docker-compose.yml`. Si al abrir `http://localhost:8081` aparece Jenkins u otro servicio, eso indica que ese puerto ya está ocupado: establece `PMA_PORT=8082` (o cualquier puerto host libre) en `.env` y ejecuta `docker compose up -d` para publicar phpMyAdmin en el puerto elegido. Si VitePress comparte 5173 con Vite, usa el puerto 5174 recomendado.

Lista los puertos y estados publicados por Compose:

```
docker compose ps
```

Almacenamiento no escribible

Comprueba desde PHP que la ruta privada está montada y permite escritura:

```
docker compose exec app sh -c 'printf "FCT_STORAGE_PATH=%s\n" "$FCT_STORAGE_PATH"; test -d "$FCT_STORAGE_PATH" &&
        test -w "$FCT_STORAGE_PATH"'
```

En Docker, `FCT_STORAGE_PATH` debe seguir siendo `/var/lib/fct/storage`; si usas XAMPP, crea el directorio local fuera de `public/` y asigna permisos a la cuenta que ejecuta Apache.

MariaDB o ClamAV aún no saludables

Consulta el estado de los servicios:

```
docker compose ps
```

Consulta los últimos mensajes de MariaDB y ClamAV para encontrar errores de inicio o el progreso de descarga de firmas:

```
docker compose logs --tail=100 db clamav
```

El primer arranque de ClamAV puede tardar; espera a que indique `healthy` antes de investigar como fallo.

Para permitir cargas de prueba locales sin escaneo, configura explícitamente ambos valores: `APP_ENV=development` y `FCT_CLAMAV_ENABLED=false`; los registros de PHP indicarán que se omitió ClamAV. El bypass también se acepta en `APP_ENV=test` y `APP_ENV=testing`. Si se requiere análisis real, establece `FCT_CLAMAV_ENABLED=true` y revisa la conectividad interna a `clamav:3310`. Nunca uses el bypass en producción.

Si la aplicación rechaza las cargas por firmas antiguas aunque el contenedor figure como `healthy`, actualiza las firmas y solicita que ClamAV las vuelva a cargar en memoria:

```
docker compose exec -T clamav freshclam --log=/tmp/fct-freshclam-update.log
docker compose exec -T clamav clamdscan --reload
docker compose exec -T clamav clamdscan --version
```

La recarga puede tardar. Comprueba la fecha mostrada por `clamdscan --version` y los registros antes de repetir la carga. El registro separado de `freshclam` evita el conflicto con el archivo de log que utiliza su proceso automático.

La página se ve sin estilos o sin islas Vue

Con `FCT_VITE_DEV_SERVER` definido, las islas y el CSS dependen de que Vite esté en marcha. Si el contenedor `frontend-dev` lleva mucho tiempo activo, puede servir una versión antigua de `frontend/src/main.ts`; reinícialo y recarga con Ctrl+F5:

```
docker compose restart frontend-dev
```

Comprueba que Vite responde con `curl.exe -s -o NUL -w "%{http_code}" http://localhost:5173/frontend/src/main.ts` (debe devolver `200`). Si no usas Vite, vacía `FCT_VITE_DEV_SERVER` en `.env` y recrea PHP con `docker compose up -d app`.

Ciclos DAW y DAM

Si no aparece DAM en una base ya existente, aplica una vez `database/migrations/009_desarrollo_aplicaciones_multiplataforma.sql` a esa base. En un volumen MariaDB nuevo, Compose ejecuta esa migración automáticamente. El matching compara IDs de especialidad: DAW sólo coincide con DAW y DAM con DAM.

Disponibilidad o controles de contraseña

Las horas se envían como `HH:MM` desde `<input type="time">` y el servidor acepta también `HH:MM:SS`; si una carga manual falla, revisa que días, indicador de disponibilidad indiferente y token CSRF correspondan al contrato. En formularios de contraseña, el botón de ojo cambia la visibilidad sin enviar el formulario; si no cambia, comprueba que `public/assets/js/password-toggle.js` esté cargado y que el campo siga siendo `type="password"` al cargar la página.

Reiniciar sin borrar datos

Reinicia los contenedores sin borrar volúmenes:

```
docker compose down
docker compose up -d --build
```

Para volver a inicializar la base de datos tras cambios de esquema o migraciones, consulta Restablecimiento de Base de Datos por Cambios de Esquema o Migraciones y revisa la advertencia sobre el alcance de `docker compose down -v`.

Más información

- Guía de Integraciones en PDF
- Guía de Integraciones editable
- Hoja de ruta de migración
- Flujo de datos y ciclo de vida
- Diagrama completo de arquitectura
- Configuración y fuentes de verdad: `routes/web.php`, `config/`, `database/` y `docker-compose.yml`.